

From Prompt Engineering to RAG — Study Guide

Source: Databricks GenAI Solution Academy, March 2025 — Topic 1.1 Prompt Engineering

1. Learning Objectives

- Describe **prompts** and **prompt engineering**.
 - Discuss techniques and best practices of prompt engineering.
 - Describe nuances between **RAG, prompt engineering, fine-tuning, and pre-training**.
 - Identify use cases where **RAG** improves quality, reliability, and accuracy of LLM completions.
 - Describe the core components of the **RAG architecture**.
 - Connect **Databricks** capabilities with the various components of RAG.
-

2. Prompt Basics — Definitions

- **Prompt:** An AI prompt is an **input or query given to a large language model (LLM)** to elicit a specific response or output.
 - **Prompt Engineering:** The practice of **designing and refining prompts** to optimize the responses generated by AI models.
-

3. Prompt Components

A good prompt usually consists of 4 parts:

Component	Meaning
Instruction	A clear directive specifying what the model should do.
Context	Background / additional information providing necessary details to understand the task.
Input / Question	The specific query or data the model must process.
Output type / Format	The desired structure or style of the generated response.

Example (all 4 components present):

"Summarize the following article and write a list of top 3 important points in markdown format that answer the following question: 'How is climate change affecting polar bear habitats and their ability to find food?' Article: <...>"

- Instruction → "Summarize the following article and write a list of top 3 important points"
 - Output format → "markdown format"
 - Question → about climate change / polar bears
 - Context → the article text
-

4. Prompt Engineering Techniques

4.1 Zero-shot Prompting

- Prompt that generates text or performs a task **without providing any examples** or additional training specific to that task.
- Example: *"Classify the following review into neutral, negative or positive: Review: '...' Sentiment: ..."*

4.2 Few-shot Prompting

- Prompt provides **a few input-output examples** to guide the model in generating the desired output.
- Example: Showing 2 classified reviews (Positive, Negative) before asking the model to classify a 3rd review.

4.3 Prompt Chaining (Break tasks into subtasks)

- **Multiple tasks are linked together**, with the output of one prompt serving as the **input for the next**.
- Allows complex tasks to be broken into manageable steps.
- Example: Step 1 — extract relevant quotes from a document using `<quotes></quotes>` tags; Step 2 — use those quotes to answer the question.

4.4 Chain-of-Thought (CoT) Prompting — *"Let's think step-by-step"*

- Enhances the **reasoning capabilities** of LLMs by guiding them to articulate their thought process step-by-step, similar to human reasoning.
- Triggered by phrases like *"Let's think step-by-step to solve this."*
- **Research findings are mixed:**
 - Some research: asking the model to think step-by-step helps solve reasoning questions.
 - Other research: models don't always follow CoT **faithfully**.
- Example (banana problem):

- Without CoT → "The answer is 27." (wrong)
 - With CoT → "23 - 20 = 3, then 3 + 6 = 9. The answer is 9." (correct)
-

5. Prompt Engineering — Tips & Tricks

5.1 Prompts are Model-Specific

- Different models may require different prompts.
- Provide **examples and cues** to guide the model's response generation.
- Different use cases may require different prompts.
- **Iterative development is key:**
 - Adjust the **temperature** parameter → **higher** for more **creative** outputs, **lower** for more **focused** responses.
- Be aware of **bias and hallucination**.

5.2 Format Prompts

- Use **delimiters** to distinguish between instruction and context:
 - Pound sign ###
 - Backticks ```
 - Braces / brackets {} / []
 - Dashes ---
- Ask the model to return **structured output**: HTML, JSON, table, markdown, etc.
- Provide a **correct example** of the output format.
 - e.g., *"Return the movie name mentioned in the form of a Python dictionary. The output should look like {'Title': 'In and Out'}"*

5.3 Guide the Model for Better Responses

- **Ask it not to hallucinate / make things up:**
 - *"Do not make things up if you do not know. Say 'I do not have that information'."*
 - **Ask it not to assume or probe for sensitive info:**
 - *"Do not make assumptions based on nationalities"*
 - *"Do not ask the user to provide their SSNs"*
 - **Ask it not to rush to a solution** (give it time to "think"):
 - Use **Chain-of-Thought for Reasoning**.
 - *"Explain how you solve this math problem."*
 - *"Do this step-by-step. Step 1: Summarize into 100 words. Step 2: Translate from English to French..."*
-

6. Benefits vs Limitations of Prompt Engineering

Benefits	Limitations
Simple and efficient — time taken to generate ideal results is significantly low.	Output depends on the model used .
Predictable results — consistently meets predefined accuracy standards.	Limited by pre-trained model's internal knowledge — for external knowledge, we need RAG .
Tailored outputs — customization of AI responses to fit specific needs or styles.	

Introduction to RAG — Study Guide

1. How do Language Models Learn Knowledge?

Three approaches, in **decreasing order of complexity and compute-intensiveness**:

Approach	What it is	Requirement
Model Pre-Training	Training an LLM from scratch	Large datasets (billions to trillions of tokens)
Model Fine-Tuning	Adapting a pre-trained LLM to specific datasets or domains	Thousands of domain-specific or instruction examples
Passing Contextual Information ← <i>focus of course</i>	Combining an LLM with external knowledge retrieval	External knowledge base; use vectors to search and provide relevant context

2. Passing Context to LMs Helps Factual Recall

- Passing **context as model inputs** improves **factual recall**. *Analogy: exam with open notes.*
- LLMs are evolving to accept a **larger/infinite input token window size**.

Downsides of "long" context:

- Higher API costs (more input tokens)
- Longer completion/inference times
- Content in the middle may be overlooked → "**Lost in the Middle**" and "**Needle in a Haystack**" test

⇒ Ongoing research to address the "**context limitation**".

3. What is RAG?

RAG = Retrieval Augmented Generation (Lewis et al., 2020)

- A **pattern** that improves the **efficacy** of LLM applications by **leveraging custom data**.
- Works by **retrieving** documents relevant to a query and providing them as context to **augment** the prompt, improving **generation**.
- **Main problem solved**: the **knowledge gap** — enhances **accuracy and relevance** of responses.

Flow: Query → Retrieve domain-specific context → Prompt (Query + Similar doc1 + Similar doc2 + ...) → LLM → Response

4. RAG Use Cases

Use Case	Description
Q&A Chatbots	Incorporate LLMs with chatbots for more accurate answers. Automates customer support and website lead follow-up.
Search Augmentation	LLMs augment search results with LLM-generated answers.
Content Creation and Summarization	Generating news articles, summarizing lengthy reports, etc.

5. Main Concepts of RAG Workflow

Component	Description
Index & Embed	Embedding model creates vector representation of documents and queries.
Vector Store	Stores unstructured data indexed by vectors — via vector DB, library, or plugin.
Retrieval	Search stored vectors using similarity search to retrieve relevant info.
Filtering & Reranking	Selecting or ranking retrieved documents before passing as context.
Prompt Augmentation	Enhance context via injection of data retrieved from the Vector Store.
Generation	LLM generates a response for the user's request.

6. Sample RAG Architecture (Step-by-Step)

Document Embedding (offline):

1. Source PDFs → Embedding Model → embeddings stored in Vector Store.

User Query (online): 2. User asks a question. 3. Embedding Model converts query to embeddings. 4. Similarity search against the Vector Store.

Generation: 5. Similar chunks/documents passed as context/memory to the LLM. 6. LLM augments and generates completion. 7. User receives an answer.

Same embedding model is used for both documents and queries so similarity comparisons are meaningful.

7. Benefits of RAG Architecture

1. **Up-to-date and accurate responses** — not based solely on static training data; uses up-to-date external sources.
2. **Reducing hallucinations** — mitigates incorrect info; outputs can include **citations of original sources** for human verification.
3. **Domain-specific contextualization** — interfaces with proprietary/domain-specific data; improves accuracy via contextually relevant info.
4. **Efficiency and cost-effectiveness** — alternative to fine-tuning; enables **in-context learning** without the up-front overhead; good when models need frequent data updates.

8. Mapping RAG Workflow to Databricks Features

Workflow Stage	Databricks Feature	Key Points
Find Relevant Context	Mosaic AI Vector Search	Built into Databricks Intelligence Platform; integrated with governance; auto-syncs with underlying Delta table .
Generate Response	Mosaic AI Model Serving	Serves Foundation Models (e.g., Llama-3) and External Models (e.g., GPT-4) via Foundation Model APIs . Mosaic AI Playground lets you interact with them.
Serve RAG Chain	Mosaic AI Model Serving	Unified interface; low-latency, high-availability; scales up/down.

Other stage-specific features mentioned: **Delta Live Tables** (batch/stream ingestion for document embedding) and **Unity Catalog's Model Registry** (for the generation model).

QUIZ QUESTIONS WITH ANSWERS

B. Technical

Q14. What is stored in a Vector Store, and what three options exist? **A: Unstructured data indexed by vectors.** It can be stored with a **vector DB, library, or plugin.**

Q15. What type of search retrieves relevant vectors? **A: Similarity search.**

Q16. Why must the same embedding model be used for documents and queries? **A: Because** similarity comparisons are only meaningful when both vectors live in the same embedding space. Different models produce incompatible vector spaces.

Q17. Databricks feature for batch/stream ingestion in document embedding? **A: Delta Live Tables.**

Q18. Databricks feature for similarity search / finding relevant context? **A: Mosaic AI Vector Search.**

Q19. Which feature serves Foundation Models (Llama-3) and External Models (GPT-4)? **A: Mosaic AI Model Serving, via Foundation Model APIs.**

Q20. What does Mosaic AI Playground allow? **A: Letting users interact with supported models** (Foundation and External Models).

Q21. Which Databricks component auto-syncs with the underlying Delta table for vector search? **A: Mosaic AI Vector Search.**

Q22. Which component governs/registers models in the generation stage? **A: Unity Catalog's Model Registry.**

Q23. Name two properties of Mosaic AI Model Serving when used to serve the RAG chain. **A: (1) Low-latency, high-availability** model deployment; (2) **Scales up or down** to meet demand changes. (Also: unified interface for deploying & querying models.)

C. Output-Based / Scenario

Q24. Identify the component responsible for:

- (a) Converting source PDFs to vectors → **Embedding Model**
- (b) Storing those vectors for fast lookup → **Vector Store**
- (c) Converting user question to vector → **Embedding Model** (same one)
- (d) Finding most relevant chunks → **Retrieval (Similarity Search)**
- (e) Reranking the top-k chunks → **Filtering & Reranking**
- (f) Generating final NL answer → **Generation Model (LLM)**

Q25. Internal HR policy document changes monthly. Fine-tuning or RAG? **A: RAG** — because the data updates frequently. RAG supports in-context learning with no up-front fine-tuning overhead, while fine-tuning would require re-training every time the policy changes.

Q26. An LLM is producing fabricated legal citations. Two RAG-based remedies? **A: (1)** Retrieve authoritative legal documents from a trusted vector store and inject them as context. **(2)** Return **citations of original sources** in the output to allow human verification.

Q27. Use case for summarizing 500-page financial reports? **A: Content Creation and Summarization.**

Q28. Dev stuffs a 200k-token PDF into a single prompt instead of using RAG. Two issues? **A: (1) Higher API costs** (more input tokens), (2) **"Lost in the middle"** — mid-document content is likely overlooked. (Also: longer inference times.)

Q29. Match each RAG stage to the Databricks feature:

- (i) Document ingestion → **Delta Live Tables**
- (ii) Find relevant context → **Mosaic AI Vector Search**
- (iii) Generate response → **Mosaic AI Model Serving** (Foundation / Custom Models)
- (iv) Register & govern generation model → **Unity Catalog's Model Registry**

D. Quiz POV (MCQ / Short Answer)

Q30. RAG stands for: a) Random Access Generation b) Retrieval Augmented Generation ✓ c) Retrieval-Aided Grounding d) Recursive Answer Generator

Q31. Main problem solved by RAG: a) High training cost b) Knowledge gap ✓ c) Slow inference d) Tokenization

Q32. Which is **NOT** a downside of long context? a) Higher API costs b) Longer inference time c) Lost in the middle d) Reduced model creativity ✓ (*not mentioned in the notes*)

Q33. A vector store specializes in storing _____ data indexed by vectors. **A: unstructured**

Q34. Least compute-intensive approach? a) Pre-training b) Fine-tuning c) Passing contextual information (RAG) ✓ d) All are equal

Q35. True/False — The same embedding model is used for documents and queries. **A: True**

Q36. Step where retrieved docs are selected or ranked: a) Retrieval b) Index & Embed c) Filtering & Reranking ✓ d) Generation

Q37. Databricks feature to find relevant context: a) Delta Live Tables b) Mosaic AI Vector Search ✓ c) Unity Catalog d) Mosaic AI Playground

Q38. NOT a benefit of RAG: a) Up-to-date responses b) Reducing hallucinations c) Eliminating the need for an LLM ✓ (*RAG still uses LLMs*) d) Domain-specific contextualization

Q39. RAG enables _____-context learning without the up-front overhead of fine-tuning. **A: in**

Q40. Match:

- 1. Find Relevant Context → **b) Mosaic AI Vector Search**
- 2. Generate Response → **a) Mosaic AI Model Serving (Generation)**
- 3. Serve RAG Chain → **c) Mosaic AI Model Serving (Deployment)**

Q41. True/False — RAG outputs can include citations of original sources. **A: True**

Q42. Example of a **foundation model** served via Mosaic AI Model Serving: a) GPT-4 (*this is listed as an external model*) b) Llama-3 ✓ c) Delta d) Iceberg

Q43. Passing context is analogous to an exam with _____. **A: open notes**

Preparing Data for RAG Solutions — Study Guide

1. Why is Data Prep Important for RAG?

Five potential issues when data is prepared improperly:

- **Poor quality model output** — inaccurate, incomplete, or biased data → misleading or incorrect responses.
- **"Lost in the middle"** — in long context, LLMs tend to overlook documents placed in the middle (*needle in haystack test*).
- **Inefficient retrieval** — poorly prepared data decreases accuracy/precision of retrieval.
- **Exposing data** — poor data governance leaks data during retrieval.
- **Wrong embedding model** — decreases quality of embeddings and retrieval accuracy.

2. Data Prep Process Overview (3 Stages)

External sources → Ingestion & Pre-processing → (1) Data Storage & Governance → (2) Chunking (Fixed-Size / Semantic) → (3) Embedding → Vector Store

3. Stage 1 — Data Storage and Governance

Delta Lake

A **unified data management layer** that brings data reliability and fast analytics to cloud data lakes. It's the optimized storage layer for the **Databricks DI Platform. Open | Fast | Reliable.**

- **Data types supported:** Structured, Semi-structured, Unstructured, Streaming.
- **Cloud storage:** Azure Data Lake Storage, Amazon S3, Google Cloud Storage.
- **Three pillars:**
 - **Data Quality** → Optimized Performance; Data Consistency and Reliability.
 - **Data Lineage** → Data Version Tracking; Data Governance.
 - **Featurization** → Feature Definition/Discovery/Sharing; Feature Publishing and Drift Monitoring.

Unity Catalog (UC)

Governance, discovery, and access control for RAG applications.

- **Unified visibility** into data and AI
- **Single permission model** for data and AI
- **Open data sharing**
- **Six capabilities:** Discovery, Access Controls, Lineage, Monitoring, Auditing, Data Sharing
- **Objects covered:** Tables, Files, Models, Notebooks, Dashboards

4. Stage 2 — Data Extraction & Chunking

Typical process: (1) Split documents into chunks → (2) Embed chunks with a model → (3) Store them in a vector store.

Constraints/Risks: chunks could be **out of context**; **no overall logic/rigor**.

How to Chunk Data?

Strategy	Description
Semantic Chunking	Chunk by sentence/paragraph/section; leverage special punctuation (e.g., ., \n); include/inject metadata/tags/titles.
Fixed-size Chunking	Divide by a specific number of tokens; simple and computationally cheap.

Chunking Strategy is Use-Case Specific

- **How long are our documents?** 1 sentence? N sentences?
- 1 chunk = 1 sentence → embeddings focus on **specific meaning**.
- 1 chunk = multiple paragraphs → embeddings capture **broader theme**.
- Consider splitting by headers.
- Use **ChunkViz** helper tool.

Two Enrichment Techniques

- **Chunk Overlap** — amount of overlap between consecutive chunks, ensuring **no contextual information is lost** between them.
- **Windowed Summarization** — a "**context-enriching**" chunking method where each chunk includes a **windowed summary** of the previous few chunks.

Query length insight: Long queries may have better-aligned embeddings to returned chunks; shorter queries can be more precise.

Advanced Chunking Strategies

- **Summarization** — sections A/B/C → summarize with LLM → store summaries in vector store.
- **Summarization with metadata** — for each chunk store: previous section summary, previous section image data, current section summary, the chunk itself, next section summary, next section image data.

Complex document issues:

- Text mixed with image
- Irregular placement of text
- Color-encoded focus (important for context)
- Charts with hierarchical information — order is critical
- Multi-column text — column order is crucial
- Keeping images with related information

Different document types:

- Preserving logical sections is hard.
- **HTML requires tag-based chunking** to preserve structure.
- **HTML tables are token-heavy** — for a 3×3 table:
 - Plain text: **11 tokens**
 - JSONL: **29 tokens**
 - HTML: **132 tokens** ⚠

General Approaches (for unstructured/complex raw text)

Approach	Libraries/Models	Features
Traditional Approach	PyMuPDF	Breaks text into raw constructs; very low-level, requires hard-coded rules.
Use a Layout Model (<i>spatial arrangement / organization of elements in a doc</i>)	Hugging Face's LayoutLMv3, doctr, PyPDF, Unstructured	Applies deep learning models built for text and context extraction.
Multi-Modal Models	OpenAI's GPT-4o, Alphabet's Gemini 1.5, plus OSS: Dolphin's Series, OpenFlamingo, Llava, OLMo	Multimodal LLMs intrinsically understand images but are still experimental.

5. Stage 3 — Embedding Model

- **Embedding** = a **numerical representation of content** (an N-dimensional vector).
- Vectors can be projected to 2D to visualize relationships (e.g., "dog" & "puppy" cluster together; "car" & "van" cluster together).

Choosing the Right Embedding Model

Data/Text properties:

- Vocabulary size (some models handle more diverse words)
- Domain/Topic (finance, medical, news, etc.)
- Text length (typical chunk/doc length)

Model capabilities:

- Multi-language support
- Embedding dimensions/size (higher dimensions = more storage cost)

Practical considerations:

- Beware of **context window limitations** — many embedding models **ignore text beyond their context window limits**.
- Privacy and cost/licensing for proprietary API-based models.

⇒ **Benchmark multiple models and pick the one with the best balance.**

Tip 1 — Choose Your Embedding Model Wisely

Decision flow: *Is your current embedding model trained on similar data as yours?*

- **Yes** → keep using it.
- **No** → choose another pre-trained embedding model.
- **No** → train or fine-tune your own embeddings on your data (e.g., fine-tune BERT).

Tip 2 — Ensure Similar Embedding Space for Queries and Documents

- **Use the same embedding model for indexing and querying.**
 - **Or** if using different models, ensure they're trained on similar data (same embedding space).
 - Example of a bad setup: movie-data embeddings + medical-literature query → gives bad results.
-

6. Data Preparation in Databricks

Unstructured Data Prep — with Databricks-managed embeddings

External sources → Ingestion (Tables/Volumes) → Document processing (1. Parsing, 2. Cleaning, 3. Chunking, 4. Featurization) using Workflows / Delta Live Tables / Notebooks → Storage (Delta Tables) → Automatic sync → Vector DB (Vector Search)

- **Databricks computes the embeddings via Model Serving** (Custom Models, External Models, Foundational Models).
- Supported models: any open or proprietary (OpenAI, Anthropic, Hugging Face, etc.).

Unstructured Data Prep — with user-managed embeddings

Same pipeline, but an **explicit Embedding stage** sits between Document Processing and Storage.

- **You compute and save the embeddings** (Workflows, DLT, Notebooks).
- Chunks, vectors & features → Storage (Delta Tables) → Auto sync → Vector Search.

Structured Data Prep — Feature Serving and Online Tables

External sources → Ingestion (Delta Table) → Feature Engineering (Workflows / DLT / Notebooks) → Storage (Delta Tables) → Automatic sync → Serving (Feature Serving, Online Tables)

QUIZ QUESTIONS WITH ANSWERS

B. Technical

Q16. Which three cloud storage systems does Delta Lake integrate with? **A:** Azure Data Lake Storage, Amazon S3, Google Cloud Storage.

Q17. What four data types can Delta Lake handle? **A:** Structured, Semi-structured, Unstructured, Streaming.

Q18. What is the key chunking rule for HTML documents, and why? **A: Tag-based chunking** must be used to preserve the logical structure of the content.

Q19. Token counts for a 3×3 table in each format? **A:** Plain text = **11 tokens**, JSONL = **29 tokens**, HTML = **132 tokens** — HTML is extremely token-heavy.

Q20. Which library is cited as the **Traditional Approach** for extraction, and what's its limitation? **A: PyMuPDF** — it breaks text into raw constructs and is very low-level, requiring **hard-coded rules**.

Q21. List the libraries in the **Layout Model** approach. **A:** Hugging Face's **LayoutLMv3**, **doctr**, **PyPDF**, **Unstructured**.

Q22. Which Multi-Modal models are mentioned? **A:** OpenAI's **GPT-4o**, Alphabet's **Gemini 1.5**, and OSS models: **Dolphin's Series**, **OpenFlamingo**, **Llava**, **OLMo**.

Q23. What defines a "layout model"? **A:** A model that understands the **spatial arrangement and organization of elements within a document**.

Q24. What four document-processing steps occur inside Databricks' unstructured data prep pipeline? **A:** (1) Parsing, (2) Cleaning, (3) Chunking, (4) Featurization.

Q25. Which Databricks tools orchestrate document processing? **A: Workflows, Delta Live Tables, Notebooks.**

Q26. Difference between **Databricks-managed** and **user-managed** embeddings in Vector Search? **A:** With **Databricks-managed**, Databricks computes embeddings automatically via Model Serving. With **user-managed**, **you compute and save the embeddings yourself** (via Workflows/DLT/Notebooks) before storing them in Delta Tables that auto-sync to Vector Search.

Q27. How does data in Delta Tables reach the Vector DB? **A:** Via **automatic sync** between Delta Tables storage and Vector Search.

Q28. What structured-data serving options does Databricks offer? **A: Feature Serving and Online Tables.**

Q29. Name a helper tool to experiment with chunk sizes. **A: ChunkViz.**

C. Output-Based / Scenario

Q30. For each symptom, identify the likely root cause from the 5 improper-data-prep issues:

- (a) Bot is citing the wrong document when user asks something in the middle of a 50-page PDF → **"Lost in the middle"**
- (b) A confidential salary table shows up in an answer it shouldn't → **Exposing data** (poor governance)
- (c) Retriever keeps returning irrelevant chunks → **Inefficient retrieval / Wrong embedding model**
- (d) Answers are factually wrong and biased → **Poor quality model output** (bad training data)

Q31. You're chunking a 10,000-word technical paper with clearly defined headers and subsections. Which chunking strategy fits best and why? **A: Semantic chunking** (by section/header) — it preserves logical boundaries, and metadata/title injection provides cleaner embeddings than an arbitrary token cut.

Q32. A RAG system indexes documents using `all-MiniLM-L6-v2` and queries using `text-embedding-ada-002`. What's wrong and how do you fix it? **A: Different embedding spaces** — similarity search will give poor results. Fix: use the **same embedding model for both** indexing and querying, or use different models **trained on similar data** producing the same embedding space.

Q33. A developer embeds medical-literature queries against a vector DB built on movie-data embeddings. What happens? **A:** Bad results — the embedding spaces don't align. Queries and documents must share the same space (same model or compatible models).

Q34. A chunk summary is injected with "previous section summary" and "next section summary" alongside the chunk itself. Which advanced chunking strategy is this? **A: Summarization with metadata.**

Q35. You want to embed screenshots of PDF slides with charts and diagrams. Which of the three general approaches fits best? **A: Multi-Modal Models** (GPT-4o, Gemini 1.5, Llava, etc.) — they intrinsically understand images, though still experimental.

Q36. Engineer says: "I'll just use PyMuPDF to extract this invoice that has multi-column layout, a logo, and a table." Likely problem? **A: PyMuPDF is low-level and requires hard-coded rules;** for complex layouts (multi-column, images, tables), a **layout model** (LayoutLMv3, Unstructured, doctrr) or a multi-modal model is more reliable.

Q37. Which chunk granularity captures **specific meaning** vs **broader theme**? **A: 1 sentence per chunk** → specific meaning. **Multiple paragraphs per chunk** → broader theme.

D. Quiz POV (MCQ / Short Answer)

Q38. Which is **NOT** one of the five data-prep issues? a) Exposing data b) Poor quality model output c) Wrong embedding model d) Too small context window ✓ (not listed)

Q39. Delta Lake's three pillars are: a) Quality, Lineage, Featurization ✓ b) Ingestion, Storage, Serving c) Extract, Transform, Load d) Discovery, Auditing, Sharing

Q40. Unity Catalog does **NOT** provide: a) Access controls b) Lineage c) Model training ✓ d) Auditing

Q41. Chunking by a fixed number of tokens is called: a) Semantic chunking b) Fixed-size chunking ✓ c) Windowed summarization d) Layout chunking

Q42. Fill in the blank: _____ ensures that no contextual information is lost between consecutive chunks. **A: Chunk overlap**

Q43. For a 3×3 table, which format uses the most tokens? a) Plain text b) JSONL c) HTML ✓ d) CSV

Q44. Which approach uses **LayoutLMv3**? a) Traditional Approach b) Layout Model ✓ c) Multi-Modal Models d) Fixed-size Chunking

Q45. Which is an example of a Multi-Modal model? a) PyMuPDF b) LayoutLMv3 c) GPT-4o ✓ d) BERT

Q46. True/False — Most embedding models will process any amount of text regardless of length. **A: False** — many embedding models **ignore text beyond their context window limits.**

Q47. For embedding models, higher embedding dimensions mean: a) Faster retrieval b) More storage cost ✓ c) Smaller vector DB d) Less accuracy

Q48. Tip 1 decision tree: if your current embedding model is **NOT** trained on similar data, you should: a) Keep using it anyway b) Choose another pre-trained model **or** train/fine-tune your own ✓ c) Switch to fixed-size chunking d) Increase chunk overlap

Q49. Fill in the blank: In Databricks-managed Vector Search, the platform automatically syncs from _____ to the Vector DB. **A: Delta Tables**

Q50. Match the pipeline type to its serving output:

- 1. Unstructured Data Prep → **b) Vector Search**
- 2. Structured Data Prep → **a) Feature Serving / Online Tables**

Q51. True/False — In user-managed embeddings, Databricks computes the embeddings for you. **A: False** — **you** compute and save the embeddings; Databricks auto-syncs only Delta Tables to Vector Search.

Q52. Which tool helps you iteratively experiment with chunk sizes? a) ChunkViz ✓ b) PyMuPDF c) LayoutLMv3 d) OLMo

Q53. Fill in the blank: A **layout model** understands the _____ **arrangement and organization** of elements within a document. **A: spatial**

LangChain - Basics — Study Guide

1. LLM Application Development (Raw API Approach)

Without LangChain, calling an LLM looks like this:

python

```
from openai import AzureOpenAI
from dotenv import load_dotenv

load_dotenv()
client = AzureOpenAI(api_version="2023-03-15-preview")
response = client.chat.completions.create(
    model="lums-gpt-4.1-mini",
    temperature=0,
    messages=[{"role": "user", "content": f"Explain {topic} in 3 short bullet points."}]
)
result = response.choices[0].message.content
```

Works for single calls, but becomes messy when you need prompts, retrieval, memory, tools, and chained logic.

2. Introduction to LangChain

- An **open-source framework** designed to **simplify the creation of applications using Large Language Models**.
- Acts as the **"glue"** that connects models (GPT-4, Claude, Llama) with **external data sources**.
- Helps manage **long-term memory and retrieval** around LLMs' finite context windows.

3. Why Do We Need LangChain?

Problem	LangChain's Role
Context Window Limits	LLMs have finite memory; LangChain manages long-term memory and retrieval.
Static Knowledge	LLMs are trained on historical data; LangChain enables RAG for real-time or private info.
Lack of Action	Standard LLMs can only talk; LangChain lets them act (search web, send emails, query DBs).
Complexity Management	Production apps need sequences of steps (Chains); LangChain provides a standard interface.

4. LangChain Modules (6 Core)

Module	Purpose
Model I/O	Managing prompts, connecting to LLMs, and parsing their outputs.
Retrieval	Interfaces with your data (PDFs, Databases) via Vector Stores.
Chains	The "sequence" logic that links multiple components together.
Agents	Logic that lets the LLM decide which tool to use for a task.
Memory	Persisting state between different interactions in a chat.
Callbacks	Logging, monitoring, and streaming the intermediate steps.

5. LangChain Components (8 Building Blocks)

Tools, Models, Example Selectors, Prompts, Vector Stores, Document Loaders, Output Parsers, Text Splitters.

6. A Simple Chain

- **Prompt Template:** "Tell me a joke about {topic}."
- **LLM:** request sent to the model (e.g., OpenAI).
- **Output Parser:** converts text response into clean JSON/string.
- Expressed in **LCEL (LangChain Expression Language)** using pipe syntax similar to Unix:

```
python
chain = prompt | model | output_parser
```

Example:

python

```
prompt = ChatPromptTemplate.from_template("Explain {topic} in 3 short bullet points.")
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"topic": "LangChain"})
print(result)
```

7. LCEL (LangChain Expression Language)

- **Modern syntax** for building chains using the **pipe (|) operator**.
- Makes flow of data **explicit and intuitive**.
- Every component is a **Runnable**: `prompt`, `model`, `function`, `retriever`, `parser`, `tool`.
- Composable into **arbitrarily complex pipelines**.
- Out-of-the-box support for: **streaming**, **async**, **batching**, **parallel execution**.
- Enables high-performance, production-ready systems easily.
- Built-in support for **parallel operations & async execution** via `RunnableParallel` — improves latency for multi-step tasks.
- Ideal for: retrieval (RAG), chatbots, reasoning pipelines, data processing.
- LangChain provides a **standard interface for multi-step AI logic**.

8. LangChain Ecosystem

- **LangSmith** — platform for **debugging, testing, and monitoring** your chains; makes them production-ready.
- **LangGraph** — extension for building **stateful, multi-actor applications** with LLMs; perfect for **complex agentic workflows where loops are required**.

9. RAG (Retrieval Augmented Generation)

- **Augments LLM knowledge** using additional data.
- Combines **retrieval + generation**.
- Handles data **not in training dataset**: private data, data after cutoff date, real-time.
- Improves **accuracy and relevancy**.
- Supports **evidence-based responses**; can reference source.

Typical flow (QA over unstructured data): Question → Embed question → Search Vector DB → Retrieve relevant docs → Prompt Template (Instructions + {Context} + {Question}) → LLM → Answer

10. Preparing and Storing Data (RAG Pipeline)

External sources (JSON, PDF, HTML, TXT, YouTube transcript...) → Document Loader → Text Splitter → Embedding Function → Vectorstore

10.1 Document Loaders

- Loaders return a **list of Documents**.
- Huge ecosystem: Arxiv, CSV, Discord, Email, EPub, EverNote, Facebook Chat, Figma, Git, GitHub, HTML, JSON, Markdown, Mastodon, MediaWiki Dump, Microsoft Word, MongoDB, ODT, Pandas DataFrame, PubMed, ReadTheDocs, Reddit, RSS, Slack, Snowflake, Telegram, URL, WhatsApp Chat, Wikipedia, XML, YouTube audio, YouTube transcripts.
- Example:

python

```
from langchain_community.document_loaders import YoutubeLoader
loader = YoutubeLoader.from_youtube_url("https://www.youtube.com/watch?v=8fEEbKJoNbU")
documents = loader.load()
```

10.2 Text Splitters — 5 Levels of Text Splitting

1. **Characters / Tokens**
2. **Recursive Character**
3. **Semantic Chunker**
4. **Document structure**
5. **Agent-like Splitting**

Helper tool: **ChunkViz** (<https://chunkviz.up.railway.app>).

RecursiveCharacterTextSplitter example:

python

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=2000, chunk_overlap=0)
```

10.3 Embeddings

- **Numerical representation** of content.
- **Vectors in high-dimensional space**.
- **Each dimension reflects an aspect**.
- **Similarity = Proximity in embedding space**.
- **Complexity is hidden** — we rely on an external provider.
- **Note:** data is **sent to the external provider**.

python

```
db = Chroma.from_documents(chunks, OpenAIEmbeddings())
```

10.4 Vectorstore

- **Stores, Searches, Retrieves** embeddings.
- Example: **ChromaDB** initialized from documents with **OpenAI embedding function**; optional **persist directory**.

Top Vectorstores (LangChain State of AI 2023):

1. Chroma, 2. FAISS, 3. Pinecone, 4. Qdrant, 5. DocArray, 6. Weaviate, 7. PostgreSQL, 8. Supabase, 9. Neo4j, 10. Redis, 11. Azure Cognitive Search, 12. Astra DB.

11. Using Data at Query Time

Question → Embed → Search Vectorstore → Retrieve relevant docs → Prompt Template (Instructions + {Context} + {Question}) → LLM → Answer

11.1 Retriever

- **Question → Embedding → distance → Relevant Documents.**
- Simple case:

python

```
retriever = db.as_retriever()
```

11.2 Multi Query Retriever (Another Retriever)

- Uses an **LLM to generate multiple variations of our question.**
- **Increases chances of finding Documents near to the question.**

python

```
from langchain.retrievers.multi_query import MultiQueryRetriever
retriever = MultiQueryRetriever.from_llm(retriever=db.as_retriever(), llm=llm)
```

11.3 Prompt / Template

- **Guides LLM output.**
- **Combines Question + Documents (context).**

python

```
from langchain.prompts import ChatPromptTemplate, PromptTemplate, HumanMessagePromptTemplate
prompt = ChatPromptTemplate(
    input_variables=['context', 'question'],
    messages=[HumanMessagePromptTemplate(prompt=PromptTemplate(
        input_variables=['context', 'question'],
        template="You are an assistant for question-answering tasks. Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.\nQuestion: {question} \nContext: {context} \nAnswer:"
    ))]
)
```

Prompt from Hub:

python

```
from langchain import hub
prompt = hub.pull("rlm/rag-prompt")
```

11.4 LLM

python

```
from langchain.chat_models import ChatOpenAI
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
llm = ChatOpenAI(streaming=True, callbacks=[StreamingStdOutCallbackHandler()], temperature=0)
```

Top LLM providers: 1. OpenAI, 2. Azure OpenAI, 3. Anthropic, 4. Hugging Face, 5. Vertex AI, 6. Fireworks.ai, 7. Ollama, 8. Amazon Bedrock.

Top OSS model providers: 1. Hugging Face, 2. Fireworks.ai, 3. Ollama, 4. LLAMA.CPP, 5. Replicate, 6. GPT4All, 7. together.ai, 8. Anyscale.

12. Putting Everything Together with LCEL

python

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

rag_chain = (
    {
        "context": retriever | (lambda docs: "\n\n".join(doc.page_content for doc in docs)),
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)

result = rag_chain.invoke(question)
```

B. Technical

Q16. Write the LCEL pipe expression for a basic prompt→model→parser chain. **A:** `chain = prompt | model | output_parser`

Q17. Which class loads a YouTube video transcript, and from which package? **A:** `YoutubeLoader` from `langchain_community.document_loaders`.

Q18. Name the 5 levels of text splitting. **A:** (1) Characters/Tokens, (2) Recursive Character, (3) Semantic Chunker, (4) Document Structure, (5) Agent-like Splitting.

Q19. Write code instantiating a `RecursiveCharacterTextSplitter` with chunk size 2000 and no overlap. **A:**

python

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=2000, chunk_overlap=0)
```

Q20. What tool helps visualize chunking? **A: ChunkViz**

(<https://chunkviz.up.railway.app>).

Q21. Write the one-liner that builds a Chroma vectorstore from chunks using OpenAI embeddings. **A:** `db = Chroma.from_documents(chunks, OpenAIEmbeddings())`

Q22. Write the one-liner that converts a vector DB into a retriever. **A:** `retriever = db.as_retriever()`

Q23. How do you build a Multi Query Retriever? **A:**

python

```
from langchain.retrievers.multi_query import MultiQueryRetriever
retriever = MultiQueryRetriever.from_llm(retriever=db.as_retriever(), llm=llm)
```

Q24. Which method pulls a pre-built prompt from the LangChain Hub? **A:**

`hub.pull("rlm/rag-prompt")` (e.g., `from langchain import hub`).

Q25. What does `StrOutputParser` do in an LCEL chain? **A:** Converts the model's raw message output into a **plain string** — typically the final step of the pipeline.

Q26. What's the role of `RunnablePassthrough()` in the final RAG chain? **A:** It **passes the input unchanged** to the next step — in the RAG chain, it forwards the original `question` while `retriever` separately produces the `context`.

Q27. Name the top 3 vectorstores and top 3 LLM providers per LangChain State of AI 2023. **A:** Vectorstores: **Chroma, FAISS, Pinecone**. LLM providers: **OpenAI, Azure OpenAI, Anthropic**.

Q28. Name four OSS model providers. **A:** Hugging Face, Fireworks.ai, Ollama, LLAMA.CPP (also Replicate, GPT4All, together.ai, Anyscale).

C. Output-Based / Scenario

Q29. Trace what this chain does step-by-step: `chain = prompt | llm | StrOutputParser()` when invoked with `{"topic": "LangChain"}`. **A:** (1) **Prompt** fills the template → "Explain LangChain in 3 short bullet points.". (2) **LLM** generates a chat response object. (3) **StrOutputParser** extracts the `.content` string from that response. Result: a plain string with 3 bullet points.

Q30. For a RAG pipeline, put these steps in order: *embedding function, document loader, text splitter, vectorstore*. **A:** **Document Loader** → **Text Splitter** → **Embedding Function** → **Vectorstore**.

Q31. A user asks a question. Walk through the 5 steps a RAG chain performs to return an answer. **A:** (1) **Embed the question**. (2) **Retrieve** similar document vectors from the vectorstore

(distance search). (3) **Inject** retrieved content into the prompt template as `{context}` along with `{question}`. (4) Send to **LLM**. (5) Parse output → final **Answer**.

Q32. Predict the output type (`str` / list of Documents / something else):

- (a) `loader.load()` → **list of Documents**
- (b) `db.as_retriever().invoke("what is X")` → **list of Documents**
- (c) `(prompt | llm | StrOutputParser()).invoke({...})` → **str**
- (d) `OpenAIEmbeddings().embed_query("hello")` → **list of floats (a vector)**

Q33. A developer uses a `RecursiveCharacterTextSplitter` with `chunk_size=2000`, `chunk_overlap=0` on a document that has tightly connected sentences across boundaries. What's the risk and how to fix it? **A: Context loss** at chunk boundaries — concepts split mid-thought will not retrieve well. Fix: **increase `chunk_overlap`** (e.g., 100–200 tokens) so consecutive chunks share context.

Q34. Your retriever keeps missing relevant docs because the user phrases questions differently each time. Which retriever should you use and why? **A: Multi Query Retriever** — it uses an LLM to generate **multiple variations of the question**, increasing the chance of matching relevant documents regardless of phrasing.

Q35. You want to debug a chain that behaves oddly in production. Which ecosystem tool do you reach for? **A: LangSmith** — it's built for debugging, testing, and monitoring chains.

Q36. You're building an autonomous research agent that loops "search → reflect → search again" until it has enough info. Which tool fits best? **A: LangGraph** — designed for **stateful, multi-actor, loop-based agentic workflows**.

D. Quiz POV (MCQ / Short Answer)

Q37. LangChain is best described as: a) An LLM b) A vector database c) An open-source framework for LLM apps ✓ d) A chunking algorithm

Q38. The pipe operator `|` in LangChain belongs to: a) Python's standard library b) LCEL ✓ c) OpenAI SDK d) ChromaDB

Q39. Which is **NOT** a LangChain module? a) Model I/O b) Retrieval c) Compiler ✓ d) Agents

Q40. Which module **persists state between different interactions** in a chat? a) Chains b) Memory ✓ c) Callbacks d) Agents

Q41. Fill in the blank: In LCEL, every component is a _____. **A: Runnable**

Q42. True/False — LCEL supports async, streaming, batching, and parallel execution out-of-the-box. **A: True**

Q43. Which ecosystem tool is for **debugging/testing/monitoring** chains? a) LangGraph b) LangSmith ✓ c) LangServe d) LangFlow

Q44. Which ecosystem tool is for **stateful, multi-actor, loop-based** workflows? a) LangSmith b) LangGraph ✓ c) LangChain Hub d) LangFuse

Q45. Fill in the blank: In embeddings, similarity = _____ in embedding space. **A: proximity**

Q46. Which is **NOT** among the 5 levels of text splitting? a) Characters/Tokens b) Recursive Character c) Dictionary splitter ✓ d) Semantic Chunker

Q47. The top vectorstore in LangChain State of AI 2023 is: a) Pinecone b) FAISS c) Chroma ✓ d) Weaviate

Q48. The top LLM provider in LangChain State of AI 2023 is: a) Anthropic b) OpenAI ✓ c) Hugging Face d) Vertex AI

Q49. `retriever = db.as_retriever()` turns a _____ into a retriever. **A: vectorstore / vector DB**

Q50. Multi Query Retriever works by: a) Querying the DB multiple times in parallel with the same question b) Using an LLM to generate multiple variations of the question ✓ c) Splitting the question into keywords d) Caching previous queries

Q51. In the final LCEL RAG chain, what does `RunnablePassthrough()` do? a) Caches the input b) Passes the input unchanged to the next step ✓ c) Converts input to JSON d) Embeds the input

Q52. True/False — With `OpenAIEmbeddings()`, the data stays entirely on your local machine. **A: False** — data is **sent to the external provider**.

Q53. In a typical RAG prompt template, which three elements are combined? **A: Instructions + {Context} + {Question}**

Q54. Match the component to its role:

- 1. Document Loader → **c) Ingests raw files into Documents**
- 2. Text Splitter → **a) Breaks text into smaller chunks**
- 3. Embedding Function → **d) Converts text to vectors**
-

4. Vectorstore → b) Stores/searches/retrieves embeddings

📖 Langchain Agents — Concise Notes

1. Chains vs Agents

Chain = fixed sequence of steps (input → process → output).

- Predefined, **deterministic** workflow (always same sequence).
- Usually one main task (summarization, QA).
- Each step = a prompt, model call, or function.
- Can be an **LCEL pipeline**: `template | model | parser`.

Agent = higher-level system that **dynamically decides** which actions/tools to use based on input.

- Flexible & intelligent — chooses actions at runtime.
- Can call multiple chains, tools, APIs, or run Python code.
- Good for **complex / multi-step** problems.
- Uses **LLM reasoning + tools** to decide next step.

2. Langchain Agents — Core Mechanics

- **Use an LLM Model to decide actions** — model determines which tool to call, what arguments to pass, and when to stop. Model can be a String (`gpt-4o-mini`) or an Object (`ChatAzureOpenAI`).
- **Run in a loop until completion** — Agent calls tool → observes result → reasons again → repeats, until a **final answer** or **iteration limit** is reached. Tools = Functions / API Calls.
- **Execute on a LangGraph runtime**:
 - Agents are created via `create_agent`.
 - Compiled into a **graph-based execution engine**.
 - **Nodes** = model, tools, middleware
 - **Edges** determine flow

3. `create_agent` Example

python

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import AzureChatOpenAI
```

```
@tool
def add(a: int, b: int) -> int:
    """ adds two numbers """
    return a + b
```

```
tools = [add]
```

```
model = <Invoke method to get a new AzureChatOpenAI Object>
agent = create_agent(model, tools)
```

```
result = agent.invoke({
    "messages": [{"role": "user", "content": "What is 12 + 7?"}]
})
print(result["messages"][-1].content)
```

- `create_agent` builds an AI agent that can think → choose the right tool → use it → continue reasoning → produce a final answer.
- **The docstring is important** — the agent reads it to decide what and how to call.

4. @tool Decorator

LangChain needs a structured way to tell the model:

- What a function does
- When it should be used
- What inputs it expects

When you add `@tool`, LangChain:

- **Registers the function as a Tool**
- Uses **type hints** to infer an input schema (**JSON schema**)
- Uses the **docstring** as the tool description shown to the LLM
- Exposes the function so agents can call it when reasoning

Custom name & description:

python

```
@tool("add_two_numbers", description="add two integers")
def method_1(a: int, b: int) -> int:
    """ adds two numbers """
    return a + b
```

```
add_two_numbers.invoke({a=10, b=5})
```

5. ReAct (Reason + Act)

- ReAct pattern = an **execution loop**: **Reason** → **Act** → **Observe** → **(repeat)** → **Final Answer**
- `@tool` + an agent that reasons about when to call that tool → **ReAct**
- The tool defines the **"Act"** capability in that loop:
 - `@tool` → "This is something the agent can do"
 - **Agent (LLM)** → decides *when* to do it
 - **Framework** → executes the function and returns the **Observation**

Visual flow: Request → Model → (Action ↔ Observation ↔ Tools) → Result

ReAct Example trace:

Thought: I should look up the user's invoice total.

Action: query_sql

Action Input: SELECT SUM(total) FROM invoices WHERE customer_id=123;

Observation: 15430.25

Thought: Now I can compute the discount.

Action: calculator

Action Input: 15430.25 * 0.1

Observation: 1543.025

Final Answer: The 10% discount is 1,543.03.

6. Tool Invocation (llm → agent → tool)

LangChain uses **Pydantic or reflection** to inspect your code and extracts:

- **Function Name** (e.g., `get_weather`)
- **Docstring** → becomes the "description" telling the LLM when to use it
- **Type Hints** → define the "parameters" (e.g., `location: str`)

LangChain converts this into a **JSON schema** (the "metadata" format model providers like OpenAI, Anthropic, Google expect).

`create_agent` provides the complete **glue (binding)** under the hood:

- Iterates tools list → converts each into JSON schema format.
- Configures `llm` with the metadata.
- You can manually control this with the `bind_tools` method.

7. Memory — Checkpointer

Problem: LLM/Agent has **no idea** about past turns by default. We want it to "memorize conversation."

Checkpointer = a **persistence layer** that automatically saves the state of a graph after each execution step (called a **superstep**).

- Responsible for "saving" the state of a **thread** so your agent can remember past interactions in a conversation.

Types:

- **In-Memory:** `InMemorySaver`
- **Database-Backed Savers:** `PostgresSaver`, `SqliteSaver`, `MongoDBSaver`, etc.

8. Context Engineering & Context

- **Context engineering** = practice of building dynamic systems that provide the **right information and tools, in the right format**, so an AI application can accomplish a task.
- In LangChain, **context object** = everything the LLM sees *in addition to* the user's current question so it can generate a correct, relevant response.

Context can include:

- Previous conversation turns (chat history)
- System instructions (e.g., "You are a SQL tutor...")
- Retrieved documents (RAG)
- Tool outputs
- User or session memory

LangChain's job = **assemble this context and inject it into the prompt** reliably.

9. Major Sources of Context in LangChain

1. **Prompt Template** — Prompt + System Instructions (**static context**)
 - e.g., System: "You are a senior SQL instructor"; User: "Explain indexes"
 - Combines static instructions with dynamic values.
2. **Chat History** — **short-term conversational context** (aka *thread-scoped memory*)
 - LangChain stores previous messages and replays them into each prompt.
 - **Turn 2 prompt** = [System] + [All previous messages] + [New user input]
 - **Problems:** Context window size / Token cost / Old messages confusing the model.
3. **RAG (Retrieved documents)** — context from an **external knowledge base**, not the chat.
 - Example: User asks "Explain Delta Live Tables constraints" → LangChain retrieves Databricks docs, internal PDFs, wiki pages → injects them.
4. **Long Term Memory** — Checkpointers (In-Memory, DB-Based, etc.)
5. **ContextSchema** — **Immutable Runtime Information** (e.g., User ID)

Windowed / summarized context (to avoid token explosion):

- **Window memory** → only last k messages
- **Summary memory** → compressed history (e.g., "Summary so far: User is learning Spark; focus on Delta Lake.")

10. ContextSchema

Defines **immutable runtime context** that is:

- Passed **at invocation time**
- Stays **constant during the run**
- **Not persisted automatically**

Typical uses: `user_id`, `tenant_id`, auth claims, feature flags, database connections, request metadata.

Code pattern:

python

```
from dataclasses import dataclass

@dataclass
class ColourContext:
    favourite_colour: str = "blue"
    least_favourite_colour: str = "yellow"

from langchain.agents import create_agent
agent = create_agent(
    model="gpt-5-nano",
    context_schema=ColourContext
)
```

11. Runtime Context ≠ LLM Context

LangChain defines **runtime context** as **local execution context (dependency injection)** — explicitly **NOT** the LLM context (prompt) nor the context window.

- **Runtime context** = data your code/tools/nodes can access during execution.
- **LLM context** = messages/variables you actually pass into the model call.
- The LLM only receives what you pass into `model.invoke(...)` (usually messages). If you never reference runtime context when building those messages, **the LLM never sees it**.

12. Runtime Context → Tools Call

Tools can access runtime context via the `ToolRuntime` parameter:

python

```
@tool
def get_favourite_colour(runtime: ToolRuntime) -> str:
    """Get the favourite colour of the user"""
    return runtime.context.favourite_colour

@tool
def get_least_favourite_colour(runtime: ToolRuntime) -> str:
    """Get the least favourite colour of the user"""
    return runtime.context.least_favourite_colour

agent = create_agent(
    model=llm,
    tools=[get_favourite_colour, get_least_favourite_colour],
    context_schema=ColourContext
)
```

13. Context — Runtime Instantiation

python

```
response = agent.invoke(  
    {"messages": [HumanMessage(content="What is my favourite colour?")]},  
    context=ColourContext(favourite_colour="green")  
)
```

- **Once created, context is immutable! It cannot be modified!**
- **Problem:** What if agents want to keep track of information per user/per thread as they learn new information? → **AgentState to the rescue!**

14. AgentState

AgentState holds two parts:

- **Messages** (sequence of chat messages: A, B, C, D...)
- **Custom Fields** — e.g., `User ID`, `Language` — **mutable** per-thread state the agent can update as it learns.

(Unlike `ContextSchema` which is immutable at runtime.)

15. Multi-Agent Systems

Why? As **task complexity increases**, **agent performance decreases** (single agent). → **Agent Decomposition** → **Task Distribution**: split into multiple specialized agents → better output quality.

Example: Event Planner (Coordinator pattern)

- **Coordinator agent** dispatches to sub-agents: **Flights, Venue, Playlist**.
- A writing pipeline example: Proposal → Research → Sources → Writing → Editing — each step handled by a specialized agent improves final quality.

16. Multi-Agent Design Pattern

1. **Define / Create all the tools** required by the system.
 2. **Create multiple agents** using `create_agent` API call — each with its own distinct list of tools.
 3. **Register each agent as a separate @tool**.
 4. **Create the main / coordinator agent** and register the sub-agents as tools.
-

Quiz Questions

B. Technical / Code-Based

Q23. What is wrong with this tool definition?

python

```
@tool
def add(a, b):
    return a + b
```

Answer: Missing type hints (needed to infer JSON schema) and missing docstring (used as description for the LLM).

Q24. Give the `@tool` decorator line that renames `method_1` to `add_two_numbers` and gives it the description "add two integers". **Answer:** `@tool("add_two_numbers", description="add two integers")`

Q25. Write the `create_agent` call to build an agent from a model `llm`, tools `[get_favourite_colour, get_least_favourite_colour]`, with a schema `ColourContext`. **Answer:**

python

```
agent = create_agent(
    model=llm,
    tools=[get_favourite_colour, get_least_favourite_colour],
    context_schema=ColourContext
)
```

Q26. How does a tool access a field like `favourite_colour` from the runtime context? **Answer:** By accepting a `ToolRuntime` parameter: `runtime.context.favourite_colour`.

Q27. How is context passed at invocation time? **Answer:**

python

```
agent.invoke(
    {"messages": [HumanMessage(content="...")]},
    context=ColourContext(favourite_colour="green")
)
```

Q28. List the 4 steps of the Multi-Agent Design Pattern. **Answer:** (1) Define/create all tools, (2) Create multiple agents using `create_agent`, each with its own distinct tool list, (3) Register each agent as a separate `@tool`, (4) Create the main/coordinator agent and register sub-agents as tools.

🔗 C. Output-Based

Q29. Given this ReAct trace, what is the **Final Answer**?

Thought: I should look up the user's invoice total.
Action: query_sql
Observation: 15430.25
Thought: Now I can compute the discount.
Action: calculator
Action Input: $15430.25 * 0.1$
Observation: 1543.025

Answer: *"The 10% discount is 1,543.03."*

Q30. After `add_two_numbers.invoke({a=10, b=5})`, what is returned? **Answer:** 15

Q31. Given:

python

```
@dataclass
class ColourContext:
    favourite_colour: str = "blue"
    least_favourite_colour: str = "yellow"

agent.invoke(
    {"messages": [HumanMessage(content="What is my favourite colour?")]},
    context=ColourContext(favourite_colour="green")
)
```

The tool `get_favourite_colour` returns `runtime.context.favourite_colour`. What will it return? **Answer:** "green" (the invocation-time value overrides the default).

Q32. If you invoke an agent with no `context=` argument and then try to call `get_favourite_colour` on the above `ColourContext`, what value is returned? **Answer:** "blue" (the default from the dataclass).

Q33. An agent has no Checkpointer configured. After a user says "I'm Seán", they send "What's my name?". What will likely happen? **Answer:** The agent won't know — without a Checkpointer, it has no persistent memory of prior interactions (as shown on the "LLM/Agent has no idea!" slide).

🔗 D. Quiz-POV Rapid Fire

- **LCEL chain syntax pattern?** → `template | model | parser`
- **`create_agent` is imported from?** → `langchain.agents`
- **`@tool` is imported from?** → `langchain.tools`
- **What does `result["messages"][-1].content` access?** → The content of the *last* message produced by the agent (the final answer).

- **True/False: A Chain can make runtime decisions about which tool to use.** → False (that's an Agent).
- **True/False: Tools = Functions/API Calls.** → True.
- **True/False: LangChain's short-term memory is also called thread-scoped memory.** → True.
- **Two mutable vs immutable containers:** → AgentState = **mutable** per-thread state;
ContextSchema = **immutable** runtime context.